

Filling holes in Class Template Argument Deduction

This paper proposes filling several gaps in [Class Template Argument Deduction](#).

Rationale

As one of us (Timur) has noted when giving public presentations on using class template argument deduction, there are a significant number of cases where it cannot be used. This always deflates the positive feelings from the rest of the talk because it is accurately regarded as artificially inconsistent. In particular, listeners are invariably surprised that it does not work with aggregate templates, type aliases, inherited constructors, or obey normal rules for calling function templates. We will show in this paper that these limitations can be safely removed. Note that some of these items were intentionally deferred from C++17 with the intent of adding them in C++20.

The following example (which also includes notation from [P1141R1](#) *) gives an idea of how much filling holes can smooth the road.

C++17 + P1141R1	Proposed
<pre>void f(Curve auto c) { Point<decltype(c.x_min())> top_left{c.x_min(), c.y_max()}; pmr::vector<typename decltype(c.ctrl_pts)::value_type> v(c.ctrl_pts.begin(), c.ctr_pts.end()); /* ... */ }</pre>	<pre>void f(Curve auto c) { Point upper_left_bound{c.x_min(), c.y_max()} pmr::vector v(c.ctrl_pts.begin(), c.ctr_pts /* ... */ }</pre>

To see that this really falls under the category of filling holes, the above code would work in C++17 if we just gave Point a two argument constructor corresponding to aggregate initialization and used vector instead of pmr::vector.

To give a “real-world” example, Mateusz Pusz recently approached us about his [Physical Units library](#), asking whether code like the following in his library could be made to deduce as expected:

```
template<Dimension D, Unit U, typename Rep> class quantity;
template<Dimension D, Unit U, typename Rep> quantity(Rep r) -> quantity<D, U, Rep>;

template<Unit U = meter, typename Rep = intmax_t>
using length = quantity<dimension_length, U, Rep>;

length d1(3);           // 3 meters
length d2(3.14);        // 3.14 meters
length<mile> d3(3);      // 3 miles
length<mile> d4(3.14);   // 3.14 miles
length<mile, float> d3(3.14); // 3.14 miles as float
```

We were pleased to discover on examination that the proposals in this paper for alias templates, partial template argument lists, and rules for aligning deduction guides with defaulted template parameters would have allowed the above code to work just as expected.

* For an in-depth discussion on how terse constraints can benefit from improved inferencing without requiring named types provided by Class Template Argument Deduction, see [P1168R0](#).

Class Template Argument Deduction for aggregates

We propose that Class Template Argument Deduction works for aggregate initialization as one shouldn't have to choose between having aggregates and deduction. This is well illustrated by the following example:

C++17	Proposed
<pre>template<class T> struct Point { T x; T y;}; // Aggregate: Cannot deduce Point<double> p{3.0, 4.0}; Point<double> p2 { .x = 3.0, .y = 4.0};</pre>	<pre>template<class T> struct Point { T x; T y;}; // Proposed: Aggregates deduce Point p{3.0, 4.0}; Point p2 { .x = 3.0, .y = 4.0};</pre>

In other words, deduction should take place simply from the arguments of a braced or designated initializer during aggregate initialization. This can be accomplished by forming an additional deduction guide for aggregates.

Algorithm

In the current Working Draft, an aggregate class is defined as a class with no user-declared or inherited constructors, no private or protected non-static data members, no virtual functions, and no virtual, private or protected base classes. While we would like to generate an aggregate deduction guide for class templates that comply with these rules, we first need to consider the case where there is a dependent base class that may have virtual functions, which would violate the rules for aggregates. Fortunately, that case does not cause a problem because any deduction guides that require one or more arguments will result in a compile-time error after instantiation, and non-aggregate classes without user-declared or inherited constructors generate a zero-argument deduction guide anyway. Based on the above, we can safely generate an aggregate deduction guide for class templates that comply with aggregate rules.

When [P0960R0](#) was discussed in Rapperswil, it was voted that in order to allow aggregate initialization from a parenthesized list of arguments, aggregate initialization should proceed as if there was a synthesized constructor. We can use the same approach to also synthesize the required additional deduction guide during class template argument deduction as follows:

1. Given a primary class template `C`, determine whether it satisfies all the conditions for an aggregate class ([`dcl.init.aggr`]/1.1 - 1.4).
2. If yes, let `T1`, ..., `TN` denote the types of the `N` elements ([`dcl.init.aggr`]/2) of the aggregate (which may or may not depend on its template arguments).
3. Form a hypothetical constructor `C(T1, ..., TN)`.
4. For every constructor argument of type `Ti`, if all types `T1 ... TN` are default-constructible, add a default argument value zero-initializing the argument as if by `Ti = {}`.
5. For the set of functions and function templates formed for [`over.match.class.deduct`], add an additional function template derived from this hypothetical constructor as described in [`over.match.class.deduct`]/1.

There is a slight complication resulting from subaggregates, and the fact that nested braces can be omitted when instantiating them:

```
struct Foo { int x, y; };
struct Bar { Foo f; int z; };
Bar bar{1, 2}; // Initializes bar.f.x and bar.f.y to 1; zero-initializes bar.z
```

In this case, we have two initializers, but they do not map to the two elements of the aggregate type `Bar`, instead initializing the sub-elements of the first subaggregate element of type `Foo`.

For complicated nested aggregates, there are potentially many different combinations of valid mappings of initializers to subaggregate elements. It would be unpractical to create hypothetical constructors for all of those combinations. Additionally, whether or not an aggregate type has subaggregate elements may depend on the template arguments:

```
template <typename T>
struct Bar { T f; int z; }; // T might be a subaggregate!
```

This information is not available during class template argument deduction, because for this we first need to deduce `T`. We therefore propose simply to avoid all of these problems by prohibiting the omission of nested braces when performing class template argument deduction.

Class Template Argument Deduction for alias templates

While Class Template Argument Deduction makes type inferencing easier when constructing classes, it doesn't work for type aliases, penalizing the use of type aliases and creating unnecessary inconsistency. We propose allowing Class Template Argument Deduction for type aliases as in the following example.

vector	pmr::vector (C++17)	pmr::vector (proposed)
<code>vector v = {1, 2, 3};</code> <code>vector v2(cont.begin(), cont.end());</code>	<code>pmr::vector<int> v = {1, 2, 3};</code> <code>pmr::vector<decltype(cont)::value_type> v2(cont.begin(), cont.end());</code>	<code>pmr::vector v = {1, 2, 3};</code> <code>pmr::vector v2(cont.begin(), cont.e</code>

`pmr::vector` also serves to illustrate another interesting case. While one might be tempted to write

```
pmr::vector pv({1, 2, 3}, mem_res); // Ill-formed (C++17 and proposed)
```

this example is ill-formed by the normal rules of template argument deduction because `pmr::memory_resource` fails to deduce `pmr::polymorphic_allocator<int>` in the second argument.

While this is to be expected, one suspects that had class template argument deduction for alias templates been around when `pmr::vector` was being designed, the committee would have considered allowing such an inference as safe and useful in this context. If that was desired, it could easily have been achieved by indicating that the `pmr::allocator` template parameter should be considered non-deducible:

```
namespace pmr {
    template<typename T>
    using vector = std::vector<T, identity_t<pmr::polymorphic_allocator<T>>>; // See p0887R0 for identity_t
}
pmr::vector pv({1, 2, 3}, mem_res); // OK with this definition
```

Finally, in the spirit of alias templates being simply an alias for the type, we do not propose allowing the programmer to write explicit deduction guides specifically for an alias template.

Algorithm

For deriving deduction guides for the alias templates from guides in the class, we use the following approach (for which we are very grateful for the invaluable assistance of Richard Smith):

1. Deduce template parameters for the deduction guide by deducing the right hand side of the deduction guide from the alias template. We do not require that this deduces all the template parameters as nondeducible contexts may of course occur in general
2. Substitute any deductions made back into the deduction guides. Since the previous step may not have deduced all template parameters of the deduction guide, the new guide may have template parameters from both the type alias and the original deduction guide.
3. Derive the corresponding deduction guide for the alias template by deducing the alias from the result type. Note that a constraint may be necessary as whether and how to deduce the alias from the result type may depend on the actual argument types.
4. The guide generated from the copy deduction guide should be given the precedence associated with copy deduction guides during overload resolution

Let us illustrate this process with an example. Consider the following example:

```
template<class T> using P = pair<int, T>;
```

Naively using the deduction guides from `pair` is not ideal because they cannot necessarily deduce objects of type `P` even from arguments that should obviously work, like `P({}, 0)`. However, let us apply the above procedure. The relevant deduction guide is

```
template<class A, class B> pair(A, B) -> pair<A, B>
```

Deducing `(A, B)` from `(int, T)` yield `int` for `A` and `T` for `B`. Now substitute back into the deduction guide to get a new deduction guide

```
template<class T> pair(int, T) -> pair<int, T>;
```

Deducing the template arguments for the alias template from this gives us the following deduction guide for the alias template

```
template<class T> P(int, T) -> P<T>;
```

A repository of additional expository materials and worked out examples used in the refinement of this algorithm is maintained [online](#).

Deducing from inherited constructors

In C++17, deduction guides (implicit and explicit) are not inherited when constructors are inherited. According to the C++ Core Guidelines [C.52](#), you should “use inheriting constructors to import constructors into a derived class that does not need further explicit initialization”. As the creator of such a thin wrapper has not asked in any way for the derived class to behave differently under construction, our experience is that users are surprised that construction behavior changes:

```
template <typename T> struct CallObserver requires Invocable<T> {
    CallObserver(T &&) : t(std::forward<T>(t)) {}
    virtual void observeCall() { t(); }
    T t;
};

template <typename T> struct CallLogger : public CallObserver<T> {
    using CallObserver<T>::CallObserver;
    virtual void observeCall() override { cout << "calling"; t(); }
};
```

C++17	Proposed
CallObserver b([]() { /* ... */ }); // OK	CallObserver b([]() { /* ... */ }); // OK
CallLogger</*???*/> d([]() { /* ... */ });	Derived d([]() { /* ... */ }); // Now OK

Note that inheriting the constructors of a base class must include inheriting all the deduction guides, not just the implicit ones. As a number of standard library writers use explicit guides to behave “as-if” their classes were defined as in the standard, such internal implementation details would become visible if only the internal guides were inherited. We of course use the same algorithm for determining deduction guides for the base class template as described above for alias templates.

Class Template Argument Deduction and partial template argument lists

Partial template argument lists can be given in Function Template Argument Deduction but surprisingly fail in Class Template Argument Deduction, a restriction we propose removing. Not only is this more consistent, but it has powerful applications that resolve longstanding difficulties in C++, such as creating associative containers with lambda comparators, as the following example shows.

C++17	Proposed
<pre>using namespace ba = boost::algorithm; set<string, ba::ilexicographic_compare> case_insensitive_strings(ba::ilexicographic_compare); // Lambda comparators are great for algorithms like sort // Why can't we use them for associative containers? set<int, /* ??? */> s([](int i, int j) { return std::popcount(i) < std::popcount(j); }); // Or container adaptors? priority_queue<Task, vector<Task>, /* ??? */> tasks([](Task a, Task b) { return a.priority < b.priority; });</pre>	<pre>using namespace ba = boost::algorithm; set<string> case_insensitive_strings(ba::ilexicographic_compare); // Lambda comparators are great for algorithms // Now with associative containers, too! set<int> s([](int i, int j) { return std::popcount(i) < std::popcount(j); }); // and container adaptors! priority_queue<Task> tasks([](Task a, Task b) { return a.priority < b.priority; });</pre>

Technical considerations

The above discussion naturally begs the question as to why partial template argument lists in Class Template Argument Deduction were not supported from the beginning. This feature was deferred until after C++17 to give further time to analyze the possibility of breaking changes when partially template argument lists were combined with default arguments or variadic templates as discussed below.

Default arguments

Consider the expression

```
vector<int>(MyAlloc())
```

As far back as C++98, this creates a `vector<int, allocator<int>>`, although it is unlikely to compile because `MyAlloc` almost certainly is not convertible to `allocator<int>`. By contrast, ordinary function template argument deduction would of course deduce `vector<int, MyAlloc>`. While this is exactly what we would want in this situation and is likely what we would have done if Class Template Argument Deduction was in C++ from the beginning, we would now have to contend with it being a breaking change.

Fortunately, we can get the benefit of the usual Function Template Argument Deduction without breaking existing code by the following rule:

- If a class has no explicit deduction guides, we propose that C++17 behavior applies where default template arguments override type inference. If the class has explicit guides, then inference overrides default arguments as in normal Function Template Argument Deduction.

This maintains the behavior of legacy C++ for classes that were designed before Class Template Argument Deduction was introduced, while at the same time allowing important examples like the above to be handled correctly (assuming [P1069R0](#), which we believe is certainly desirable. Of course, once a class has deduction guides, nearly any desired behavior can be specified, so other examples should be handleable as needed.

Notes:

- As this rule is only relevant for template parameters with default initializers. For class template parameters without default initializers, the programmer can ignore it.
- Without this rule, no combination of traditional deduction guides could produce this behavior, severely limiting the ability to use deduction guides to customize class behavior.
- If deduction guides are ever supported for function templates in addition to class templates as proposed in [P1168R0](#), having deduction guides follow the usual rules for function template argument deduction in which inference overrides default values seems clearly the right approach. This proposal will be consistent with that.

Variadic arguments

One other technical issue discussed during C++17 was around variadic templates. In particular, what should `tuple<int>(1, 2.3)` deduce? Again, the C++17 behavior of `tuple<int>` is incorrect, as such templates have the wrong number of arguments. While it is tempting to deduce additional arguments just like `make_tuple<int>(1, 2.3)` does, we do not propose at this time allowing additional variadic arguments to be deduced for the following reasons:

- The following code is both valid and useful in C++17

```
tuple<int> t(5, allocator<int>());
```

However, if additional variadic arguments were deduced, this code would erroneously deduce `tuple<int, allocator<int>>`, introducing a breaking change.

- Since programmers are used to thinking of `tuple<int>` as the name of a type, allowing it to deduce to the very different type `tuple<int, allocator<int>>` might well be confusing.
- We are not aware of any truly compelling use cases for deducing extra variadic arguments on a par with the ones listed above, so the downsides outweigh the benefits in our opinion.

Note:

If at some point in the future it is desired to align more closely with function templates by allowing additional variadic arguments to be deduced, it may become possible with other related language changes. For example, if the right-hand side of a deduction guide could be a *type-id* instead of just a *simple-template-id*, a metafunction could compute the deduce different `tuple` types based on whether the last argument is an allocator as well as unrelated use cases such as Richard Smith's delayed forwarding example in P1021R0, but as there is absolutely no requirement that this be considered for C++20, so we do not pursue this any further at this time.